

An Unsupervised Based Enhanced Anomaly Detection Model Using Features Importance

Nishant Sushmakar

Department of Applied Mathematics
Delhi Technological University
New Delhi, India
nishantsushmakar@gmail.com

Shivang Gupta

Department of Applied Mathematics
Delhi Technological University
New Delhi, India
shivanggupta23@gmail.com

Naman Oberoi

Department of Applied Mathematics
Delhi Technological University
New Delhi, India
naman.oberoi@outlook.com

Anshul Arora

Department of Applied Mathematics
Delhi Technological University
New Delhi, India
anshul15arora@gmail.com

Abstract—Anomaly Detection, one of the important use cases of cybersecurity, allows us to know the out of distribution behavior of processes in the network which can help us to flag any kind of attack or issue on the network. Several works have been proposed in the literature that aim to detect anomalies based on supervised algorithms. However, they fail to detect zero-day attacks. Keeping this limitation in mind, in this work, we have proposed an anomaly detection model based upon features' importance. A newly introduced dataset called BETH is used for our anomaly detection research and we outperform the baseline scores using the existing Principal Component Analysis, Histogram and Clustering based approaches which performed really well with 0.96, 0.95 and 0.94 ROC-AUC scores respectively on the test set by selecting the features extracted through different approaches such as difference in metric and assignment of weights.

Keywords—Anomaly Detection, Feature Selection, Principal Component Analysis, NSL-KDD, BETH, DARPA

I. INTRODUCTION

Cybersecurity has been a very pressing issue now that data has become the most precious thing that an enterprise possesses. The actors carrying out the attacks could have malevolent intent or just a purpose to pester. These attacks could be external or internal which makes it more difficult to detect and counteract. These attacks possess unknown risks and have the potential to steal, destroy, or modify assets of the enterprises. As a result, we have to find ways to reduce and eventually exterminate any risk of cyber-attacks.

Many methods and techniques have come to the fore trying to solve this problem. Supervised techniques like Isolation Forest, Support Vector Machines (SVM), Density-Based Spatial Clustering of Applications with Noise (DBSCAN) were used in many papers as the base methods but these techniques were found inconsequential while identifying zero day attacks which cause the maximum damage. These methods include both supervised and unsupervised techniques but the

one method that takes the most attributes into consideration is Anomaly Detection. Anomaly detection is a process that detects events and/or observations that differ from the expected behavior of a dataset. A typical data might reveal significant situations, such as a technical fault, or prospective possibilities, such as malicious activity.

As we wanted to test our technique, we looked upon many different data sets like NSL-KDD and DARPA but these datasets which are used majorly for research purposes have become outdated and give results which are inconsequential in these modern times. Therefore, we tested our technique on the BETH dataset. The BPF-extended tracking honeypot (BETH) dataset is the first cybersecurity dataset for uncertainty and robustness benchmarking. BETH is applicable for contemporary real-world deployments because it contains modern host activity as well as activity obtained from cloud services. Furthermore, some datasets contain artificial user activity, whereas BETH only contains real user activity. BETH is also one of the rare datasets that includes both kernel-process and network logs, giving it a comprehensive picture of malicious activity. To check the significance of the features in the dataset, we have calculated the importance of these features by assigning weights and finding the difference in the AUC scores by dropping them one by one. The calculated results were then used to select different subsets of the dataset on which we performed our analysis to determine the most significant features for the purpose of anomaly detection.

The rest of the paper is structured as follows. We review the related work in Section II. We discuss the datasets and features used in our methodology in Section III and IV respectively. Further, in Section V we explain the proposed methodology in detail. We review the results obtained from the proposed work in Section VI and conclude our work with future work directions in Section VII.

II. RELATED WORK

In this section, we review the related work in the field of cyber security and anomaly detection. Some of the works such as [31] - [38] have been proposed in the literature to detect anomalies / intrusions in mobile environments. However, we do not aim to detect anomalies in mobile networks. Hence, we discuss the related work in the field of anomaly detection in desktop environment.

Das et al. [2] discussed the current state of cybersecurity and analyzed how anomaly detection relies on data rather than machine learning algorithms. The J48 Decision tree performed well on the intrusion detection dataset, according to their results. They concluded that due to the rapid changes in the network, modern anomaly detection requires the most recent data to train. Highnam et al. [1] introduced BETH dataset with two types of sensor log information present in the data and evaluated baseline scores using ensemble models for anomaly detection, where isolation forest outperformed every other model. This dataset has numerous advantages over other datasets, such as the ability to answer issues about cyber security other than out of distribution analysis. The BETH dataset contains (recently recorded) genuine data with labels, unlike logs within real deployed systems that do not contain labels for malicious activities.

On the CIC-IDS 2017 dataset, Aksu et al. [3] employed deep learning to detect port scan attempts with 7 hidden layers that dramatically improved the results compared to SVM, demonstrating that malicious detection may be improved further using Deep Learning. Elmrabit et al. [4] compared the Deep Learning and Machine Learning algorithms on three different datasets to see how significant the differences were. Surprisingly, Random Forest performed well on all three datasets, emphasizing that the choice of algorithm for the task is dependent on the data generated by the system. Anomaly detection is utilized across a wide range of cybersecurity sectors, including web applications (Gao et al. [5] and Sun et al. [8]), network infrastructure (Han et al. [6]), and IoT devices (Liu et al. [7]). Researchers used to follow a typical pattern of first defining normal user behavior using machine learning algorithms, then measuring the divergence of the test data to assess whether it was an abnormality. When dealing with zero-day attacks, this is critical because the signatures of these attacks are unknown and unrecorded.

Following this technique, researchers delved into supervised and unsupervised algorithms with a goal to acquire an excessive accuracy and occasional fake advantageous rate. Some generally used algorithms protected KMeans, DBScan, SVM and Isolation Forest. Pu et al. [9] used a singular hybrid technique to categorize anomalies primarily based totally on Subspace Clustering (SSC) and One Class Support Vector Machine (OCSVM) which beat the baseline rankings of algorithms including KMeans and DBScan. The important component for detecting anomalies is to have nicely processed records and computationally cheaper algorithms. Baptista et al. [10] supplied a brand-new technique for appearing malware

evaluation the usage of self-organizing incremental neural networks or SOINN for its evaluation and detection. The technique performed nicely for .pdf and .doc but poorly for .exe and .htm documents which might be the primary supply of attacks.

Siddiqui et al. [11] had used Isolation Forest for detection of attacks and then presented the reasons as to why the algorithm had identified some computers as anomalous. This was then reviewed by an analyst who provided the feedback for these anomalous attacks as signals. But human input makes the whole system slow and dispensable. Meira et al. [12] uses several unsupervised learning algorithms on NSL-KDD and ISCX dataset and evaluates them with respect to different scores. Unsupervised algorithms are better at detecting zero-day attacks but the datasets used are outdated and don't satisfy the modern needs. Hariharan et al. [21] used a streamlined approach to detect threats in real time by using various anomaly models to produce an outlier score on a multitude of data using elastic search, which increased the efficiency of real time detection by a great extent. Feature selection has been an important factor to increase the performance in Rashid et al. [22], for which they have reduced the size of the training dataset by using a subset of features which brought down the time complexity of training the model. Evangelou et al. [23] defined individual device behavior and have taken it as the normal state to distinguish anomalies from usual behavior by using Quantile Regression Forest for anomaly detection. Ferrag et al. [24] worked on a comparative study of different deep learning approaches namely deep descriptive discriminative models and generative/unsupervised models that were able to detect anomalies efficiently on two different datasets: CSE-CIC-IDS2018 and BoT-IoT.

Le et al. [26] explored the use of four unsupervised anomaly detection algorithms and its ensemble on the CERT insider threat Dataset. Sharma et al. [27] used Long Short-Term Memory (LSTM) based Autoencoder to model the behavior of a user based on their activities during a session and thus identify the anomalous data points. A threshold is then defined in both the papers to separate the outliers from benign data records. Merrill et al. [28] defined a modified approach of Autoencoder by incorporating cumulative error scoring, early stopping via knee detection and percentile loss, to improve the scoring for anomaly detection. Even though variational autoencoders provide good results, they are computationally time consuming and a two stage method is proposed by Neuschmied et al. [29] to combine a filtering method with variational autoencoder using a reconstruction probability.

III. DATASETS

In the consequent subsections, we highlight the datasets.

A. NSL-KDD

This dataset contains the information of processes in a network traffic and a target value for whether the processes are anomaly or not in both CSV (Comma-separated values) as well as ARFF (Attribute Relation File Format). NSL-KDD

is the successor of the KDD dataset. The KDD Cup 1999 dataset included a large amount of redundant records, leading learning algorithms to be weighted toward frequent records and thus preventing them from learning less frequent records, which are generally more destructive to networks. Also NSL-KDD is the modified version of a dataset which was created in 1999 but now due to the technological Advancement this dataset does not meet the modern requirements.

B. DARPA 1998

The 1998 DARPA Intrusion Detection Evaluation had two components: an off-line evaluation and a real-time evaluation. The off-line evaluation used network traffic and audit logs collected on a simulated network to test intrusion detection systems. To try to identify attack sessions during the process of operations, this data was made to process in batch mode by the systems. The Air Force Research Laboratory (AFRL) received intrusion detection devices for real-time testing. These devices were installed in the AFRL network test bed and sought to detect attack sessions in real time while conducting routine tasks. Intrusion detection systems were put to the test as part of the off-line, real-time, or both evaluations.

C. BETH Dataset

The BETH dataset now contains 8,004,918 events collected from 23 honeypots over the course of around five hours on a major cloud provider. They chose the first subset of the process logs for benchmarking and discussion. This subset was then divided into training, validation, and testing sets, with a 60/20/20 split based on host, number of logs created, and activity logged—only the test set contains an assault. Two sensor logs make up the dataset: kernel-level process calls and network traffic. Only process logs are included in the first benchmark subset. Each process call has 14 raw features as well as two hand-crafted labels. Network traffic is included in the KDD-1999 and NSL-KDD datasets, whereas limited process calls are included in the DARPA 1998 dataset. These datasets, on the other hand, are at best almost a decade old and are collected on in-house servers. BETH includes modern host activity as well as data from cloud services, making it useful in today's world.

IV. FEATURES

Now, we explain the features which we have used in our proposed model. The BETH Dataset comprises of 16 columns. 14 columns are attributes for which pre-processing has been done as per [1] and we have engineered several other features with it to include in the set of 12 features. Remaining 2 columns are the target variables which denote whether the record is suspicious and/or evil. The 14 attributes are discussed as follows:

timestamp - This feature has been left out as we are not doing time series analysis.

processId - Process ID 0,1 and 2 denote values used by the OS. Hence a binary mapping is done here to identify whether the process is used by the OS.

threadId - This attribute did not contribute to the analysis hence it is ignored.

parentProcessId - It is similar to the processId and a mapping similar to processId has been done.

userId - For OS activity, userId is assigned a value of less than 1000. Hence a binary mapping has been done here to identify whether it is an OS activity or a user driven activity.

mountNamespace - This determines the access of the process to mount points. Most common value found was for mnt/ directory which is 4026531840 or 0xF0000000. This value was common for all logs with userId $\geq$ 1000. Hence a binary mapping was done to determine whether mountNamespace was 4026531840.

processName - This field was label encoded as it contains a unique set of values related to the process, this string field can be easily modified by the attackers but we have included this in the model for further scrutiny.

hostName - This denotes the honeypot from which the activity was generated. It is label encoded for the evaluation.

eventId - For each event, Linux assigns a value to it. We use this in our analysis.

eventName - Since this maps directly to eventId, this feature was label encoded.

argsNum - This feature has been used as it is.

returnValue - This feature tells us the exit status of a call and whether it was successfully completed. It has been mapped as follows :

-1 for negative values denoting there is some error

0 remains as it is denoting successful execution

1 for positive values which indicate both success and signaling to something of the parent process.

stackAddresses - This feature was binary encoded on the basis whether it contains information or not as it could not be used without an effective processing method as the value is large and difficult to relate during manual analysis.

args - This feature has been ignored for the simplicity of the model

V. METHODOLOGY

Several intrusion detection based research works have focused on developing models which can provide a high score for a chosen metric. However, we have taken a different approach by focusing on the importance of the features and finding the best combination to train the models on. This becomes important as it eliminates the features producing noise in the dataset and not dropping the features which can give us significant information on detecting anomalies.

To understand the importance of different features and how they affect different anomaly models we have worked on two different approaches based on AUC difference. The base results for the difference are taken from 4 high performing algorithms namely, Histogram Based Outlier , Cluster Based Local outlier detection, Isolation Forest and Principal Component Analysis. Figure 1 summarizes the proposed work. In the consequent subsections, we explain the methodology in detail.

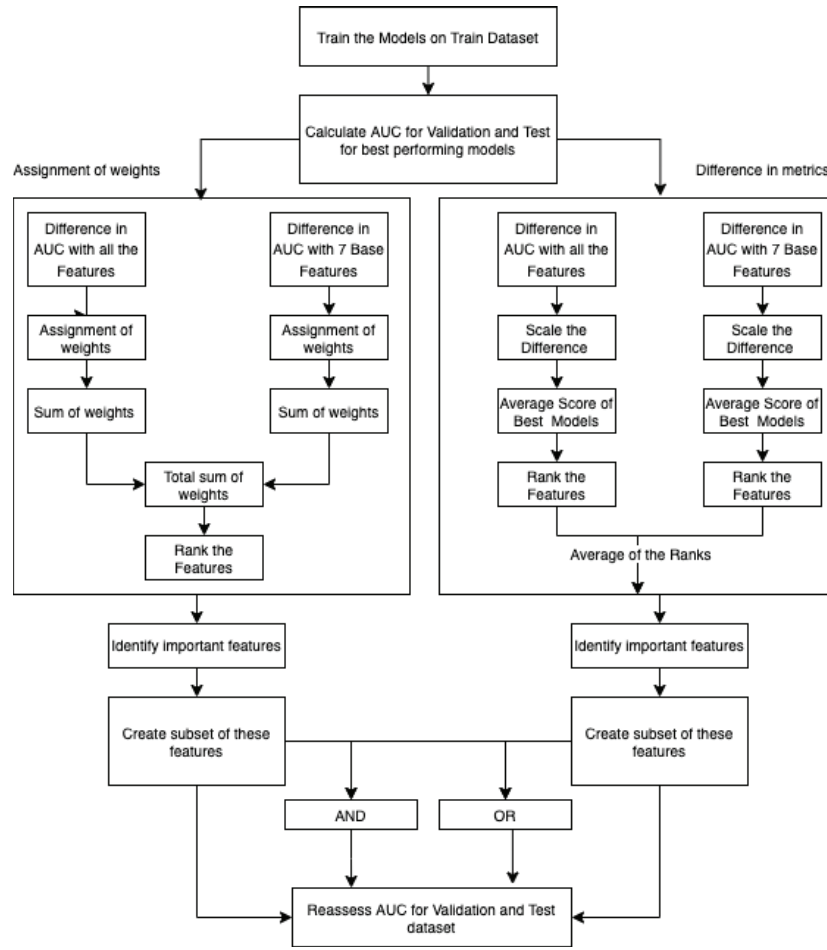


Fig. 1. Flowchart of the methodology

A. Evaluation Metric

The AUC ROC stands for Area Under Curve for Receiver Operating Characteristic curve. It is a commonly used performance indicator that is used in classification problems, plotted by finding the true positive rate and false positive rate on y and x axis respectively. A higher AUC denotes that the model predicts the classes more accurately i.e. identify 0 when label is 0 and 1 when label is 1. In case of this dataset, it means identifying whether the record is suspicious or not.

B. Algorithms

- 1) **Isolation Forest:** Isolation Forest is an unsupervised technique which isolates a single data point from the population and the less number of splits required to isolate, more are the chances of the point being an outlier. The idea behind Isolation Forests is that anomalies are data points that are rare and unusual.
- 2) **Histogram-based Outlier Score:** It is an unsupervised distance-based technique. A good choice for addressing global outliers, but not so much for treating local oddities. A global outlier is a data point that is significantly different from the rest of the data. The local oddity, on the other hand, is a data point that is near the data's

limit but is positioned at an acceptable distance from its immediate surroundings. The algorithm presupposes that the characteristics are totally independent of one another (zero multicollinearity).

3) Principal Component Analysis (PCA):

Principal component analysis calculates principal components and uses them to make changes to the database. It is basically used for dimensional reduction and visualization of the data in lesser dimensions. The weighted sum of the projected distances of the eigen hyperplanes was taken as the measure of the outlier score.

4) Minimum Covariance Determinant (MCD):

It is an estimator of robust nature which calculates multivariable positions and variances. Since the estimation of the covariance matrix is the basis of many multivariate statistical methods, MCD is an essential component in developing a robust multivariate method. It also serves as a convenient and efficient tool for detecting outliers.

5) K Nearest Neighbors:

It is a supervised technique which operates on calculating spatial distance and averaging out scores of all the nearest neighbors to perform outlier detection. It is not parametric and is widely used in real-world scenarios.

TABLE I
VALIDATION AND TEST SCORES OF ALGORITHMS

Algorithm	Validation Score	Test Score
k Nearest Neighbors (use the distance to the kth nearest neighbor as the outlier score)	0.523	0.956
Local Outlier Factor	0.534	0.956
Histogram-based Outlier Score	0.982	0.922
Clustering-Based Local Outlier Factor	0.827	0.908
Isolation Forest	0.919	0.826
Principal Component Analysis (PCA)	0.999	0.958
Outlier Detection with Minimum Covariance Determinant (MCD)	0.853	0.573

6) Local Outlier Factor:

This is an unsupervised anomaly detection technique that calculates local density deviations in relation to adjacent data points for a particular data point.

7) Clustering-Based Local Outlier Factor:

In the specific Local Outlier Detection method the sparse/small clusters are treated as outliers. Outliers are detected using clustering-based techniques, which extract the link between Objects and Cluster.

Table I represents the scores of anomaly detection algorithms on the 7 features which were used in the related work in [1] for the base model.

C. Feature Importance

Every feature provides vital information to the model, the more critical the information is, higher is its importance. In supervised learning there are several different techniques to identify the critical features such as Permutation importance, SHapley Additive exPlanations (SHAP) values and Tree based Importance. We did not use supervised learning because different supervised learning techniques give different feature importance results due to the nature of the algorithm. Extreme gradient boosting follows a Breadth first search approach while Light gradient boosting machine follows a Depth first search approach which can impact the entropy while making a tree and can result in assigning different importance to features. Anomaly Detection does not have any respective approach to identify the importance of the features, therefore we have analyzed the importance of the features using two different methods. While checking the different features for importance args feature was removed from further analysis because the args consist of different sub-parts which were having a high variability in its value and when we applied Natural Language Processing to extract the count of the features it adds up to high dimensionality with no further increase in performance.

1) *Assignment of Weights:* If after dropping a feature, the drop in AUC score is significant, then we assign a weight of 0 for the dropped feature and if the score improves, then we assign a weight of 1. There were cases where the AUC score for the dropped feature was not significant and in the vicinity of the base score. In such cases a weight of 0.5 was assigned. A sum of these values were taken for different models for both validation and test score. Lower value of sum or higher value

TABLE II
FEATURE IMPORTANCE AS PER WEIGHTS

Features	Base score for 7 features as per [1]	Base Score as per all features	Sum	Rank	Is this feature important ?
processId	6	6	12	1	×
threadId	6.5	4	10.5	2	×
parentProcessId	6	3	9	7	✓
userId	1	1	2	12	✓
mountNamespace	5	4.5	9.5	6	✓
processName	5	2.5	7.5	10	✓
hostName	6.5	4	10.5	2	×
eventId	5.5	4.5	10	5	×
eventName	5	2.5	7.5	10	✓
stackAddresses	6	4.5	10.5	2	×
argsNum	5.5	3.5	9	7	✓
returnValue	4.5	4	8.5	9	✓

of rank denotes that the feature is important while a higher value of sum or lower value of rank means that the feature could possibly be dropped. The validation and test score was calculated by training on all 12 features and then dropping them one by one. Two different base scores were taken, one as per [1] when we consider the selected 7 features and one as per all the 12 available features as described in Table II.

2) *Differences in metric:* This method monitors the drop in the AUC score when a feature is not taken into account for training and then the base value is checked with an initial set of 12 features and also [1] the 7 selected features by the author. The following steps are done for all the features on 4 selected high performing models with each set of base values on validation and test data and then we aggregate the score using the average for validation and test and then we do an average for the models and then standardize the scores and then we take a rank depending on the scores of the features where higher scores and rank means more importance of the features. The results in Table III are the same for when base taken with 12 features as well as for 7 features.

TABLE III
FEATURE IMPORTANCE USING DROP IN AUC SCORES

Features	Importance Score with Base Features	Rank	Is this feature important ?
processId	0.205	2	×
threadId	0.075	1	×
parentProcessId	0.270	5	×
userId	0.889	12	✓
mountNamespace	0.363	9	✓
processName	0.336	6	✓
hostName	0.254	4	×
eventId	0.348	8	✓
eventName	0.448	10	✓
stackAddresses	0.254	3	×
argsNum	0.343	7	✓
returnValue	0.481	11	✓

TABLE IV
FEATURE SELECTION FOR DIFFERENT SUBSETS OF TRAINING DATA

Features	As per [1]	Considering 12 features	Assignment of weights	Difference in metric	AND	OR
processId	✓	✓	×	×	×	×
threadId	×	✓	×	×	×	×
parentProcessId	✓	✓	✓	×	×	✓
userId	✓	✓	✓	✓	✓	✓
mountNamespace	✓	✓	✓	✓	✓	✓
processName	×	✓	✓	✓	✓	✓
hostName	×	✓	×	×	×	×
eventId	✓	✓	×	✓	×	✓
eventName	×	✓	✓	✓	✓	✓
stackAddresses	×	✓	×	×	×	×
argsNum	✓	✓	✓	✓	✓	✓
returnValue	✓	✓	✓	✓	✓	✓
timestamp	×	×	×	×	×	×
args	×	×	×	×	×	×

TABLE V
AUC SCORES FOR THE SUBSETS OF THE FEATURES

	Histogram Based Outlier Score		Clustering-Based Local Outlier Factor		Isolation Forest		Principal Component Analysis	
Subset/Scores	Validation	Test	Validation	Test	Validation	Test	Validation	Test
As per [1]	0.982	0.922	0.827	0.908	0.919	0.826	0.999	0.958
Considering 12 features	0.992	0.928	0.993	0.602	0.944	0.839	0.999	0.958
Assignment of weights	0.982	0.958	0.498	0.924	0.926	0.905	0.999	0.963
Difference in metric	0.872	0.966	0.505	0.927	0.597	0.925	0.999	0.964
AND	0.872	0.968	0.240	0.923	0.612	0.920	0.999	0.964
OR	0.982	0.957	0.506	0.927	0.926	0.917	0.999	0.963

VI. RESULTS AND INTERPRETATION

In this section, we review the results obtained from the proposed approach.

Table IV represents the baseline features and features in different subsets as per feature importance. Further two more subsets have been created based on AND and OR operation of the subsets generated by the two feature selection techniques i.e., Assignment of weights and difference in metrics.

Based on Table IV, we see significant improvement of results in the subset generated as per assignment of weights and OR subset. On looking at the features used in these subsets, processName and eventName provide more information than taking the processId and eventId as originally proposed by [1]. Other than that, the features proposed in [1] have been proven of importance as per the above techniques. By considering all the 12 features, we are overfitting the model and hence obtaining higher validation scores but lower testing scores compared to the previous two subsets.

Table V represents the validation and test scores of the best performing algorithms on the subsets of training data which has been found by calculating the feature importance. The baseline model is that as mentioned in [1]. It is evident from the numbers that there is a significant improvement in the

test results of the new subsets. Validation scores have fallen which may be due to the fact that the model was overfitting earlier. Subsets generated as per assignment of weights and the OR subset have produced comparable results and are the best subsets to test the model on. PCA has outperformed every other model and also on different subset of features have only improved the result in validation and test set for PCA which have degraded the results for other different models in different subsets.

VII. CONCLUSION AND FUTURE WORK

In this work, we proposed an anomaly detection model based upon supervised approach and features' importance. Based on the review of various anomaly detection works and implementation of machine learning techniques on the BETH Dataset, we have successfully outperformed the baseline score as proposed in the work done in [1]. Further we have established ways to identify features which would yield more information and accordingly choose a subset which would help in identifying anomalies more accurately.

We plan to further incorporate args features in our model which holds potentially invaluable information which has been dropped from our training set. We will try to automate this work so it can be replicated by other researchers.

REFERENCES

- [1] K. Highnam, K. Arulkumaran, Z. Hanif, and N. Jennings, "BETH Dataset: Real Cybersecurity Data for Anomaly Detection Research", in ICML Workshop on Uncertainty and Robustness in Deep Learning, 2021.
- [2] R. Das and T. H. Morris, "Machine Learning and Cyber Security", 2017 International Conference on Computer, Electrical and Communication Engineering (ICCECE), pp. 1-7, 2017.
- [3] D. Aksu, and M. Aydin, "Detecting Port Scan Attempts with Comparative Analysis of Deep Learning and Support Vector Machine Algorithms", in International Congress on Big Data, Deep Learning and Fighting Cyber Terrorism, pp. 77-80, 2018.
- [4] N. Elmrabit, F. Zhou, F. Li and H. Zhou, "Evaluation of Machine Learning Algorithms for Anomaly Detection", 2020 International Conference on Cyber Security and Protection of Digital Services (Cyber Security), pp. 1-8, 2020.
- [5] Y. Gao, Y. Ma, and D. Li, "Anomaly detection of malicious users' behaviors for web applications based on web logs", 2017 IEEE 17th International Conference on Communication Technology (ICCT), 2017.
- [6] M. Han, and I. Kim, "Anomaly detection method using network pattern analysis of process", 2015 World Congress on Internet Security (WorldCIS), 2015.
- [7] Z. Liu, N. Thapa, A. Shaver, K. Roy, X. Yuan, and S. Khorsandroo, "Anomaly Detection on IoT Network Intrusion Using Machine Learning", 2020 International Conference on Artificial Intelligence, Big Data, Computing and Data Communication Systems (icABCD), 2020.
- [8] Y. Sun, Y. Xie, W. Wang, S. Zhang, Y. Wu, and J. Feng, "RPAD: An Unsupervised HTTP Request Parameter Anomaly Detection Method", 2020 IEEE 19th International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom), 2020.
- [9] G. Pu, L. Wang, J. Shen and F. Dong, "A hybrid unsupervised clustering-based anomaly detection method", Tsinghua Science and Technology, 26(2), 146-153, 2021.
- [10] Irina Baptista, Stavros Shialeles and Nicholas Kolokotronis, "A Novel Malware Detection System Based on Machine Learning and Binary Visualization", 1-6, 2019.
- [11] M. A. Siddiqui, "Detecting Cyber Attacks Using Anomaly Detection with Explanations and Expert Feedback", ICASSP 2019 - 2019 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), pp. 2872-2876, 2019.
- [12] Jorge Meira, Rui Andrade, Isabel Praça, João Carneiro, Verónica Bolón-Canedo, Amparo Alonso-Betanzos and Goreti Marreiros, (2020). "Performance evaluation of unsupervised techniques in cyber-attack anomaly detection", Journal of Ambient Intelligence and Humanized Computing.
- [13] Andrew Golczynski and John A. Emanuella, "End-To-End Anomaly Detection for Identifying Malicious Cyber Behavior through NLP-Based Log Embeddings", arXiv preprint arXiv:2108.12276, 2021.
- [14] Mondther Alabadi and Yuksel Celik, "Anomaly Detection for Cyber-Security Based on Convolution Neural Network : A survey", 1-14, 2020.
- [15] T. T. Teoh, G. Chiew, E. J. Franco, P. C. Ng, M. P. Benjamin and Y. J. Goh, "Anomaly detection in cyber security attacks on networks using MLP deep learning", 2018 International Conference on Smart Computing and Electronic Enterprise (ICSCEE), pp. 1-5, 2018.
- [16] Jan vávra and Martin Hromada and Ludek Lukas and Jacek Dworzecki, "Adaptive Anomaly Detection System Based on Machine Learning Algorithms in an Industrial Control Environment", International Journal of Critical Infrastructure Protection, 34, 100446, 2021.
- [17] Nath Das, Mausumi and Tapalina Bhattasali, (2020). "ANOMALY DETECTION USING MACHINE LEARNING APPROACHES", Azerbaijan Journal of High Performance Computing, 3, 196-206, 2020.
- [18] I. Guyon and A. Elisseeff, "An introduction to variable and feature selection", in Journal of machine learning research, vol. 3, pp. 1157-1182, 2003.
- [19] T. Subbulakshmi and A. F. Afroze, "Multiple learning based classifiers using layered approach and Feature Selection for attack detection", in Emerging Trends in Computing, Communication and Nanotechnology (ICE-CCN), pp. 308-314, 2013.
- [20] M. Nour, and J. Slay, "The evaluation of Network Anomaly Detection Systems: Statistical analysis of the UNSW-NB15 data set and the comparison with the KDD99 data set", in Information Security Journal: A Global Perspective, vol. 25, no. 3, pp. 18-31, 2016.
- [21] A. Hariharan, A. Gupta, T. Pal, "CAMLPAD: Cybersecurity Autonomous Machine Learning Platform for Anomaly Detection", Advances in Intelligent Systems and Computing, vol 1130. Springer, Cham, 2020.
- [22] A. N. M. Bazlur Rashid, Mohiuddin Ahmed, Leslie F. Sikos, and Paul Haskell-Dowland, "Anomaly Detection in Cybersecurity Datasets via Co-operative Co-evolution-based Feature Selection", ACM Trans. Manage. Inf. Syst. 13, 3, Article 29 (September 2022), 39 pages, 2022.
- [23] Marina Evangelou, Niall M. Adams, "An anomaly detection framework for cyber-security data", Computers and Security, Volume 97, 101941, ISSN 0167-4048, 2020.
- [24] Mohamed Amine Ferrag, Leandros Maglaras, Sotiris Moschoyiannis, Helge Janicke, "Deep learning for cyber security intrusion detection: Approaches, datasets, and comparative study", Journal of Information Security and Applications, Volume 50, 2020, 102419, ISSN 2214-2126, 2020.
- [25] M. Ahmed, A. Naser Mahmood and J. Hu, "A survey of network anomaly detection techniques", Journal of Network and Computer Applications, 60, 19-31, 2016.
- [26] D. C. Le and N. Zincir-Heywood, "Anomaly Detection for Insider Threats Using Unsupervised Ensembles", in IEEE Transactions on Network and Service Management, vol. 18, no. 2, pp. 1152-1164, June 2021.
- [27] Balaram Sharma, Prabhat Pokharel, and Basanta Joshi, "User Behavior Analytics for Anomaly Detection Using LSTM Autoencoder - Insider Threat Detection", In Proceedings of the 11th International Conference on Advances in Information Technology (IAIT2020). Association for Computing Machinery, New York, NY, USA, Article 5, 1-9, 2020.
- [28] N. Merrill and A. Eskandarian, "Modified Autoencoder Training and Scoring for Robust Unsupervised Anomaly Detection in Deep Learning", in IEEE Access, vol. 8, pp. 101824-101833, 2020.
- [29] H. Neuschmied, M. Winter, K. Hofer-Schmitz, B. Stojanovic, and U. Kleb, "Two Stage Anomaly Detection for Network Intrusion Detection", In Proceedings of the 7th International Conference on Information Systems Security and Privacy - ICISPP, ISBN 978-989-758-491-6; ISSN 2184-4356, pages 450-457, 2021.
- [30] Dipankar Dasgupta, Zahid Akhtar, and Sajib Sen, "Machine learning in cybersecurity: a comprehensive survey", The Journal of Defense Modeling and Simulation (2020): 1548512920951275, 2020.
- [31] A. Arora, S. Garg and S. K. Peddoju, "Malware Detection Using Network Traffic Analysis in Android Based Mobile Devices", 8th International Conference on Next Generation Mobile Apps, Services and Technologies, pp. 66-71, 2014.
- [32] A. Arora, S. K. Peddoju and M. Conti, "PermPair: Android Malware Detection Using Permission Pairs", in IEEE Transactions on Information Forensics and Security, vol. 15, pp. 1968-1982, 2020.
- [33] A. Arora and S. K. Peddoju, "Minimizing Network Traffic Features for Android Mobile Malware Detection", Proceedings of the 18th International Conference on Distributed Computing and Networking, Article 32, pp. 1-10, 2017.
- [34] A. Arora and S. K. Peddoju, "NTPDroid: A Hybrid Android Malware Detector Using Network Traffic and System Permissions", 17th IEEE International Conference On Trust, Security And Privacy In Computing And Communications/ 12th IEEE International Conference On Big Data Science And Engineering (TrustCom/BigDataSE), pp. 808-813, 2018.
- [35] K. Khariwal, J. Singh and A. Arora, "IPDroid: Android Malware Detection using Intents and Permissions", 4th World Conference on Smart Trends in Systems, Security and Sustainability (WorldS4), pp. 197-202, 2020.
- [36] A. Arora, S. K. Peddoju, V. Chouhan, and A. Chaudhary, "Hybrid Android Malware Detection by Combining Supervised and Unsupervised Learning", In Proceedings of the 24th Annual International Conference on Mobile Computing and Networking, pp. 798-800, 2018.
- [37] M. Upadhayay, A. Sharma, G. Garg and A. Arora, "RPNDroid: Android Malware Detection using Ranked Permissions and Network Traffic", 5th World Conference on Smart Trends in Systems Security and Sustainability (WorldS4), pp. 19-24, 2021.
- [38] G. Garg, A. Sharma, and A. Arora, "SFDroid: Android Malware Detection Using Ranked Static Features", in International Journal of Recent Technology and Engineering, vol. 10, no. 1, pp. 142-152, 2021.